

《数据库系统》课内实验指导书

——实验 7：数据库事务与备份、恢复

一、实验目的

- 1) 熟悉事务的编程接口与操作，掌握并发控制与事务隔离机制；
- 2) 理解数据备份与恢复的核心策略与实现流程，强化数据安全与容灾意识；
- 3) 通过体验“数据管理”和“数据安全”模块的核心实践环节，旨在让学生掌握保障数据正确性、一致性和可用性的关键技术。

二、实验要求

- 1) 根据实验内容，通过部署的 openGauss 数据库系统完成题目练习；
- 2) 提交内容：一份 txt 文本（包含题目、SQL 语句）+一份 word 实验报告（包含题目、SQL 语句、必要截图和文字说明，小结部分可以体现心得或课程建议）；
- 3) 提交时间：6.5 晚上之前；
- 4) 整合在一个压缩包里，文件命名为：姓名-班级-学号-实验 7。

三、实验内容

- 1) 方案一：导入指定的样本数据库 emloy.sql，完成指定题目的练习。
- 2) 方案二：自行选择样本数据库和设计练习题目。【需体现事务、备份与恢复的核心机制】

从两个方案中选择一个作为本次实验内容。

四、数据库事务一实践练习

1.1 事务语法

事务是数据库操作的基本单位，保证一组操作要么全部执行，要么全部不执行。确保数据的一致性和完整性，在并发或故障时维护数据正确。

事务控制语句的语法和含义：

语法	含义
START TRANSACTION / BEGIN	表示事务开始执行。
COMMIT	表示事务完成所有数据操作，并且保存操作结果，标志着事务的成功完成。
ROLLBACK / END	表示事务未完成所有数据操作，重新返回到事务开始，标志着事务的撤销。

1.2 事务示例

例 1、启动事务，使用 SELECT 语句查询 Person 表内容，再使用 INSERT 语句插入新数据，最后提交事务，接着再次查询 Person 表内容。

```
-- 创建 Person 表
CREATE TABLE Person (
  id INT PRIMARY KEY,
  name VARCHAR(255)
);
```

```
-- 启动事务
BEGIN;
-- 查询 Person 表内容
SELECT * FROM Person;
-- 插入新数据
INSERT INTO Person (id, name) VALUES (1, 'Alice');
-- 提交事务
COMMIT;

-- 再次查询 person 表内容
SELECT * FROM Person;
```

没插入新数据之前的 person 表内容如下图所示：

```
db1=# db1=# id | name
-----+-----
(0 rows)
```



插入新数据并提交了事务的 `person` 表内容如下图所示：

```
db1=# SELECT * FROM Person;
 id | name
----+-----
  1 | Alice
(1 row)
```

例 2、启动事务，再使用 `SELECT` 语句查询 `person` 表内容，接着插入 `id` 为 122 的新数据，使用 `ROLLBACK` 语句进行回滚。最后再次查询 `person` 表内容。

```
-- 启动事务
BEGIN;
-- 查询 person 表内容
SELECT * FROM Person;
-- 插入 id 为 122 的新数据 (name 列可空)
INSERT INTO Person (id) VALUES (122);
-- 回滚事务
ROLLBACK;

-- 再次查询 person 表内容
SELECT * FROM Person;
```

插入新数据前的 `person` 表内容如图所示：

```
db1=# SELECT * FROM Person;
 id | name
----+-----
  1 | Alice
(1 row)
```

回滚事务之后，再次查询 `person` 表内容，发现与事务执行前的结果是相同的：

```
db1=# INSERT INTO Person (id) VALUES (122);
INSERT 0 1
db1=# ROLLBACK;
ROLLBACK
db1=# SELECT * FROM Person;
 id | name
----+-----
  1 | Alice
(1 row)
```

1.3 事务保存点语法

事务保存点可以提供细粒度的回滚控制，设置保存点后，可针对事务内的局部错误进

行调整，继续执行后续操作，减少事务开销。

事务保存点语句的语法和含义：

语法	含义
SAVEPOINT savepoint_name	在当前事务里建立新的保存点， savepoint_name表示保存点的名称。
ROLLBACK TO SAVEPOINT savepoint_name	事务回滚到保存点savepoint_name。
RELEASE SAVEPOINT	删除一个保存点。

1.4 事务保存点示例

例 1、开启事务，对 `person` 表进行查询，然后插入新数据，建立保存点 `s1`，然后再插入新数据，建立保存点 `s2`，然后回滚到保存点 `s1`。最后提交事务，再次查询 `person` 表内容。

```
-- 开启事务
BEGIN;
-- 第一次查询 person 表
SELECT * FROM person;
-- 插入新数据（假设插入 id 为 3, name 为 'InsertedBeforeS1'）
INSERT INTO person (id, name) VALUES (3, 'InsertedBeforeS1');
-- 建立保存点 s1
SAVEPOINT s1;
-- 再次插入新数据（假设插入 id 为 4, name 为 'InsertedAfterS1'）
INSERT INTO person (id, name) VALUES (4, 'InsertedAfterS1');
-- 建立保存点 s2
SAVEPOINT s2;
-- 回滚到保存点 s1（撤销保存点 s1 之后的插入操作）
ROLLBACK TO SAVEPOINT s1;
-- 提交事务
COMMIT;

-- 再次查询 person 表
SELECT * FROM person;
```

第一次查询 `person` 表内容结果如图所示，此时还没执行任何插入操作：

```
db1=# BEGIN;
BEGIN
db1=# SELECT * FROM person;
 id | name
----+-----
  1 | Alice
(1 row)
```

第二次查询 `person` 表内容结果如图所示，发现只插入了保存点 `s1` 前的新数据，说明事务回溯到保存点 `s1`：



```
db1=# SELECT * FROM person;
id |      name
----+-----
 1 | Alice
 3 | InsertedBeforeS1
(2 rows)
```

例 2、开启事务，对 `person` 表进行查询，然后插入新数据，建立保存点 `s1`，然后再插入新数据，接着删除保存点 `s1`，再建立新的保存点 `s1`，然后回滚到新的保存点 `s1`。接着再次查询 `person` 表内容，最后提交事务。

```
BEGIN;
-- 第一次查询 person 表
SELECT * FROM person;
-- 插入新数据（第一次插入）
INSERT INTO person (id, name) VALUES (5, 'insert5');
-- 建立保存点 s1
SAVEPOINT s1;
-- 再次插入新数据（第二次插入）
INSERT INTO person (id, name) VALUES (6, 'insert6');
-- 删除保存点 s1
RELEASE SAVEPOINT s1;
-- 重新建立保存点 s1（此时标记第二次插入后的状态）
SAVEPOINT s1;
-- 回滚到新的保存点 s1（因无后续操作，不撤销任何插入）
ROLLBACK TO SAVEPOINT s1;
-- 再次查询 person 表（会显示两次插入后的数据）
SELECT * FROM person;
-- 提交事务（永久保存两次插入的数据）
COMMIT;
```

第一次查询 `person` 表内容结果如图所示，此时还没执行任何插入操作：

```
db1=# BEGIN;
BEGIN
db1=# SELECT * FROM person;
id |      name
----+-----
 1 | Alice
 3 | InsertedBeforeS1
(2 rows)
```

第二次查询 `person` 表内容结果如图所示，说明事务回滚到了新的保存点 `s1`，执行了两次插入操作：

```
db1=# db1=# id | name
-----+-----
 1 | Alice
 3 | InsertedBeforeS1
 5 | insert5
 6 | insert6
(4 rows)
```

1.5 事务管理（并发控制与隔离）

事务管理：通过并发控制技术和隔离级别定义，在允许事务并发执行的同时，将其相互影响控制在可接受范围内，最终确保事务的隔离性（ACID 中的 “I”）和一致性（ACID 中的 “C”）。

针对同一个数据库，事务并发处理时的数据不一致性问题：

- 1) 脏读：一个事务读取到另一个事务未提交的更新数据。
- 2) 不可重复读：在一个事务内，多次读取同一数据却得到不同结果。（例：事务 A 读数据 data，之后事务 B 修改了 data 并率先提交，事务 A 提交前再读 data。针对同一个 data 两次读的结果不一样。）

隔离级别：

- 1) READ COMMITTED（默认级别）：读提交。

脏读	√（解决）	不可重复读	×（未解决）
----	-------	-------	--------

- 2) REPEATABLE READ：可重复读。

脏读	√（解决）	不可重复读	√（解决）
----	-------	-------	-------

准备工作：创建一个 account 表

```
-- 创建测试表
CREATE TABLE account (
    id INT PRIMARY KEY,
    balance DECIMAL(10, 2)
);

-- 插入初始数据
INSERT INTO account VALUES (1, 1000), (2, 2000);
```

例 1、隔离级别设置为 READ COMMITTED（默认级别），验证解决“脏读”。

事务A:

```
BEGIN;  -- 默认隔离级别为 READ COMMITTED
SELECT balance FROM account WHERE id = 1;  -- 第一次读: 1000
```

事务B:

```
BEGIN;  -- 默认隔离级别为 READ COMMITTED
UPDATE account SET balance = 1500 WHERE id = 1;
```

事务A:

```
SELECT balance FROM account WHERE id = 1;  -- 第二次读: 1000 (未脏读, 若读出1500则为脏读)
COMMIT;
```

但会产生“不可重复读”问题:

事务A:

```
BEGIN;  -- 默认隔离级别为 READ COMMITTED
SELECT balance FROM account WHERE id = 1;  -- 第一次读: 1000
```

事务B:

```
BEGIN;
UPDATE account SET balance = 1500 WHERE id = 1;
COMMIT;  -- 提交事务
```

事务A:

```
SELECT balance FROM account WHERE id = 1;  -- 第二次读: 1500 (不可重复读)
COMMIT;
```

例 2、隔离级别设置为 REPEATABLE READ，验证解决“不可重复读”问题。

事务A:

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM account WHERE id = 1;  -- 第一次读: 1000
```

事务B:

```
BEGIN;
UPDATE account SET balance = 1500 WHERE id = 1;
COMMIT;  -- 提交事务
```

事务A:

```
SELECT balance FROM account WHERE id = 1;  -- 第二次读: 1000 (结果一致, 不可重复读解决)
COMMIT;
```



1.6 数据库事务的练习

(1) 在 `employees` 表中, 为员工 `emp_no` 为 10003 的员工新增一条工作经历记录到 `dept_emp` 表, 同时在 `titles` 表中为其添加一个新的职位记录。请编写一个事务, 确保这两个操作要么都成功提交, 要么都不生效, 并通过 `SELECT` 给出事务提交前后查询相关数据的 SQL 语句及预期结果。

(2) 需要完成一个事务, 要对 `salaries` 表中 `emp_no` 为 10001 的员工薪资进行调整, 先将薪资提高 20%, 然后模拟一个错误操作 (如将 `to_date` 设置为一个不合理的过去时间), 使用事务回滚确保数据库数据不受错误操作影响。请编写完整的事务代码, 并给出事务回滚前后查询该员工薪资数据的 SQL 语句及预期结果。

(3) 在 `departments` 表中, 要对部门信息进行一系列操作。首先插入一个新部门记录, 设置保存点; 接着尝试修改一个不存在的部门编号 (模拟错误操作); 然后回滚到保存点, 再插入另一个新部门记录, 最后提交事务。请编写完整的事务代码, 并给出各个关键步骤 (设置保存点、回滚到保存点、提交事务) 前后查询 `departments` 表数据的 SQL 语句及预期结果。

(4) 基于 `employ` 数据库, 设置两个用户连接, 同时并发对同一个数据库 (`employ` 数据库) 进行操作, 设置实验进行下列验证:

- READ COMMITTED (默认级别) 下不会发生脏读。
- READ COMMITTED 下会发生不可重复读。
- 设置隔离级别 REPEATABLE READ, 解决不可重复读问题。

五、数据库备份与恢复—实践练习

2.1 逻辑备份与恢复

逻辑备份: 通过软件技术导出数据库中的逻辑对象 (如表、索引、存储过程等), 将数据内容写入特定格式的输出文件, 形成数据内容的映像。

例 1、利用 `opengauss` 提供的工具进行逻辑备份与恢复。

(1) 创建数据库 `db1`、创建用户 `user1`, `db1` 所属为 `user1`; 为 `user1` 分配“管理员”权限:

`ALTER ROLE user1 SYSADMIN;` (一定要授予管理员权限!!)

(2) 创建模式 `ll`, 创建学生表 `ll.student`

```
CREATE TABLE ll.student(  
    id INT,  
    name VARCHAR(30)  
);
```

(3) 创建存储备份文件的文件夹

指令: `mkdir -p /home/opengauss/logical/backup`

(4) 使用 `gs_dump` 工具进行逻辑备份:

- 以目录形式备份数据库 `db1` 全部数据, 导出为目录 `db1`:



```
gs_dump -U user1 -w "123456abc." -p 5432 -F d -f /home/opengauss/logical/backup/db1 db1
```

```
[opengauss@2128c69e6b7e /]$ gs_dump -U user1 -W "123456abc." -p 5432 -F d -f /home/opengauss/logical/backup/db1 db1
gs_dump[port='5432'][db1][2025-04-29 23:31:31]: The total objects number is 432.
gs_dump[port='5432'][db1][2025-04-29 23:31:31]: [100.00%] 432 objects have been dumped.
gs_dump[port='5432'][db1][2025-04-29 23:31:31]: dump database db1 successfully
gs_dump[port='5432'][db1][2025-04-29 23:31:31]: total time: 3167 ms
```

b) 以自定义形式备份学生表，导出为 student.dump 文件：

```
gs_dump -U user1 -w "123456abc." -F c -t ll.student -f /home/opengauss/logical/backup/student.dump db1
```

```
[opengauss@2128c69e6b7e /]$ gs_dump -U user1 -W "123456abc." -F c -t ll.student -f /home/opengauss/logical/backup/student.dump db1
gs_dump[port='5432'][db1][2025-04-29 23:33:25]: The total objects number is 420.
gs_dump[port='5432'][db1][2025-04-29 23:33:25]: [100.00%] 420 objects have been dumped.
gs_dump[port='5432'][db1][2025-04-29 23:33:25]: dump database db1 successfully
gs_dump[port='5432'][db1][2025-04-29 23:33:25]: total time: 2141 ms
```

(5) 使用 gs_dumpall 工具进行逻辑备份：

- 直接使用 gs_dumpall 工具会报错：

```
gs_dumpall: query failed: ERROR: permission denied for relation pg_authid
DETAIL: N/A
```

- 原因：权限不够。
- 解决方案：为用户授予“管理员”权限。
- 指令：**ALTER ROLE user1 SYSADMIN;**（在 gsql 界面执行）

a) 以文本方式导出所有数据库所有数据，导出为 gs_all.bak 文件：

```
gs_dumpall -U user1 -w "123456abc." -f /home/opengauss/logical/backup/gs_all.bak
```

```
[opengauss@2128c69e6b7e /]$ gs_dumpall -U user1 -W "123456abc." -f /home/opengauss/logical/backup/gs_all.bak
gs_dump[port='5432'][dbname='db1'][2025-04-29 23:34:45]: The total objects number is 432.
gs_dump[port='5432'][dbname='db1'][2025-04-29 23:34:45]: [100.00%] 432 objects have been dumped
gs_dump[port='5432'][dbname='db1'][2025-04-29 23:34:45]: dump database dbname='db1' successfully
gs_dump[port='5432'][dbname='db1'][2025-04-29 23:34:45]: total time: 2890 ms
```

b) 以文本方式仅导出数据库定义，导出为 gs_define.bak 文件：

```
gs_dumpall -U user1 -w "123456abc." -s -f /home/opengauss/logical/backup/gs_define.bak
```

```
[opengauss@2128c69e6b7e /]$ gs_dumpall -U user1 -W "123456abc." -s -f /home/opengauss/logical/backup/gs_define.bak
gs_dump[port='5432'][dbname='db1'][2025-04-29 23:35:55]: The total objects number is 430.
gs_dump[port='5432'][dbname='db1'][2025-04-29 23:35:55]: [100.00%] 430 objects have been dumped
gs_dump[port='5432'][dbname='db1'][2025-04-29 23:35:55]: dump database dbname='db1' successfully
gs_dump[port='5432'][dbname='db1'][2025-04-29 23:35:55]: total time: 3198 ms
```

注意：如果仅导出数据（不包括定义），导出到 dat.bak 文件：

```
gs_dumpall -U user1 -w "123456abc." --data-only -f /home/opengauss/logical/backup/dat.bak
```



(6) 使用 `gs_restore` 工具进行逻辑恢复，将导出的目录 `db1` 恢复成数据库 `db1`:

a) 删除数据库 `db1`:

```
DROP DATABASE db1;
```

b) 使用 `gsql` 命令连接 `db1` 数据库，发现失败:

```
[opengauss@2128c69e6b7e /]$ gsql -d db1 -p 5432
gsql: FATAL: database "db1" does not exist
```

c) 创建数据库 `db1` 后，使用 `gs_restore` 工具再恢复数据库 `db1`:

```
CREATE DATABASE db1 OWNER user;
```

```
[opengauss@2128c69e6b7e /]$ gs_restore -U user1 -W "123456abc." -d db1 -F d /home/opengauss/log
ical/backup/db1
start restore operation ...
table student complete data imported !
table person complete data imported !
Finish reading 10 SQL statements!
end restore operation ...
restore operation successful
total time: 71 ms
```

d) 使用 `gsql` 命令连接 `db1` 数据库，使用 `select` 语句可发现数据库已经完全恢复:

```
[opengauss@2128c69e6b7e /]$ gsql -d db1 -p 5432
gsql ((openGauss 5.0.0 build ) compiled at 2023-04-18 09:47:39 commit 0 last mr )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

db1=# select * from ll.student;
 id | name
----+-----
(0 rows)
```

(7) 使用 `gs_restore` 工具进行逻辑恢复，将导出的学生表 `student.dump`(自定义格式) 恢复:

a) 首先删除学生表，使用 `select` 语句可发现学生表已被删除:

```
DROP TABLE ll.student;
```

b) 将导出的表格文件 `student.dump`(自定义格式) 导入到 `db1` 数据库:

```
[opengauss@2128c69e6b7e /]$ gs_restore -U user1 -W "123456abc." -F c -d db1 /home/opengauss/log
ical/backup/student.dump
start restore operation ...
table student complete data imported !
Finish reading 4 SQL statements!
end restore operation ...
restore operation successful
total time: 47 ms
```

c) 使用 `select` 语句可发现学生表已经完全恢复:

```
db1=# select * from ll.student;
 id | name
----+-----
(0 rows)
```



2.2 物理全量增量备份与恢复

物理备份：以磁盘块为单位，直接拷贝数据库的物理文件（如数据文件、控制文件、日志文件等），不考虑数据的逻辑内容。

物理全量备份：本质是对数据库在某一时刻的完整复制，备份包含该时刻数据库的全部数据（如数据文件、控制文件等物理文件）。**首次备份，建议使用全量备份。**

物理增量备份：本质是仅记录自上次备份（全量或增量备份）后数据库发生变化的数据。

例 1、利用 opengauss 提供的工具进行物理备份与恢复。

(1) 创建存放备份文件的文件夹

指令：`mkdir -p /home/opengauss/physical/gs_bak`

(2) 全量备份：使用 `gs_probackup` 工具进行全量备份。

a) 初始化备份目录，即在备份目录下创建 `backups/` 和 `wal/` 子目录，分别用于存放备份文件和 WAL 文件：

```
gs_probackup init -B /home/opengauss/physical/gs_bak
```

```
[opengauss@2128c69e6b7e /]$ gs_probackup init -B /home/opengauss/physical/gs_bak
INFO: Backup catalog '/home/opengauss/physical/gs_bak' successfully initied
```

b) 向备份目录添加备份实例。因为 `gs_probackup` 可以在一个备份目录中存储多个数据库集群的备份，所以需要将此次备份是对哪一个数据库集群进行的备份映射到一个备份实例上，无论是该数据库集群的哪一种备份都映射到这个备份实例上，接下来的完全备份、增量备份都映射到了 `gs_bak_inst` 这个实例上，将来恢复的时候也要指定从这个实例中进行恢复：

```
[opengauss@2128c69e6b7e /]$ gs_probackup add-instance -B /home/opengauss/physical/gs_bak -D /var/lib/opengauss/data --instance=gs_bak_inst
INFO: Instance 'gs_bak_inst' successfully initied
```

c) 使用 `ls` 命令查看备份目录的 `backups` 子目录和 `wal` 子目录，发现都创建了 `gs_bak_inst` 子目录：

指令：`ls /home/opengauss/physical/gs_bak/backups`

指令：`ls /home/opengauss/physical/gs_bak/wal`

d) 对数据库集群执行全量备份：

```
gs_probackup backup -B /home/opengauss/physical/gs_bak --instance=gs_bak_inst -b FULL
```

```
INFO: Backup files are synced, time elapsed: 1s
INFO: Validating backup SVHLS3
INFO: Backup SVHLS3 data files are valid
INFO: Backup SVHLS3 resident size: 659MB
INFO: Backup SVHLS3 completed
```

e) 执行 `show` 命令查看全量备份文件：

```
gs_probackup show -B /home/opengauss/physical/gs_bak --instance=gs_bak_inst
```

Instance	Version	ID	Recovery Time	Mode	WAL Mode	TLI	Time	Data	WAL
Zratio	Start LSN	Stop LSN	Type	Status					
=====									
gs_bak_inst	9.2	SVHLS3	2025-04-30 00:00:05+08	FULL	STREAM	1/0	6s	643MB	16MB
1.00	0/30000A8	0/3000268	FILE	OK					



(3) 增量备份：使用 `gs_probackup` 工具进行增量备份。

a) 若要使用增量备份功能，需要在 `db1` 数据库中修改参数 “`enable_cbm_tracking`”：

```
ALTER SYSTEM SET enable_cbm_tracking=on;
```

b) 新建测试表 `test1`：

```
CREATE TABLE test1(a int);
```

c) 对数据库 `db1` 执行第一次增量备份：

```
gs_probackup backup -B /home/opengauss/physical/gs_bak --instance=gs_bak_inst -b PTRACK
```

INFO: Validating backup SVHMJ9

INFO: Backup SVHMJ9 data files are valid

INFO: Backup SVHMJ9 resident size: 545MB

INFO: Backup SVHMJ9 completed

可能报错：

若第一次增量备份报错：ERROR: query failed: ERROR: could not find valid CBM file that contains the merging start point 00000000/05000028 query was: SELECT.

原因：CBM 文件损坏，应重新执行全量备份重建 CBM。

方法 1：bug，重置所有操作（重新开启一个 `opengauss` 容器），过段时间重新跑一遍流程。

方法 2：再进行一次全量备份，然后重启 `opengauss`，重新执行增量备份。

d) 执行 `show` 命令查看增量备份文件：

```
===== 备份文件ID ===== 说明是增量备份文件
gs_bak_inst 9.2 SVHMJ9 2025-04-30 00:16:22+08 PTRACK STREAM 1/1 6s 529MB 16MB
1.00 0/E000028 0/E0001E8 FILE OK
```

e) 新建测试表 `test2`：

```
CREATE TABLE test2(b int);
```

f) 对数据库 `db1` 执行第二次增量备份，命令参见（d），执行 `show` 命令查看增量备份文件：

```
gs_probackup backup -B /home/opengauss/physical/gs_bak --instance=gs_bak_inst -b PTRACK
```

```
gs_probackup show -B /home/opengauss/physical/gs_bak --instance=gs_bak_inst
```

```
gs_bak_inst 9.2 SVHOAO 2025-04-30 00:54:26+08 PTRACK STREAM 1/1 6s 257MB 16MB
1.00 0/10000148 0/10000308 FILE OK
```

g) 新建测试表 `test3`：

```
CREATE TABLE test3(c int);
```

(4) 全量恢复：使用 `gs_probackup` 工具进行全量恢复。

a) 打开 Docker Desktop 客户端，点击 `openGauss` 数据库中的 “INSPECT” 选项，从 “PG_DATA” 行可查看 `openGauss` 数据库数据存放路径：



```
"PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",  
"EXEC_GOSU=gosu-amd64",  
"GOSU_VERSION=1.12",  
"PGDATA=/var/lib/opengauss/data"
```

b) 删除整个数据库目录:

```
sudo rm -rf /var/lib/opengauss/data
```

c) 对数据库集群执行全量恢复:

```
gs_probackup restore -B /home/opengauss/physical/gs_bak --instance=gs_bak_inst -i SVFTW3
```

“之前全量备份文件的 ID，可通过 show 命令查看所有备份文件信息”

报错: ERROR: cannot create directory "/var/lib/opengauss/data": Permission denied.

原因: opengauss 不是 omm/root 用户, 没有权限

解决方法: 修改目录所有者和组为 opengauss: sudo chown -R opengauss:opengauss

/var/lib/opengauss, 赋予目录权限: sudo chmod -R 750 /var/lib/opengauss

INFO: Restored backup files are synced, time elapsed: 0

INFO: Restore of backup SVHMGG completed.

d) 重启 openGauss 数据库后, 测试数据库正常:

```
db1=# select * from ll.student;
```

```
id | name
```

```
-----
```

(5) 增量恢复: 使用 gs_probackup 工具进行增量恢复。

a) 执行 show 命令查看增量备份文件 id:

```
gs_probackup show -B /home/opengauss/physical/gs_bak --instance=gs_bak_inst
```

b) 执行第一次增量恢复:

因为恢复功能需要数据库目录为空, 所以需要先删除整个数据库目录:

```
sudo rm -rf /var/lib/opengauss/data
```

```
gs_probackup restore -B /home/opengauss/physical/gs_bak --instance=gs_bak_inst -i SVHMJ9
```

“第一次增量备份的文件 ID”

INFO: Restored backup files are synced, time elapsed: 0

INFO: Restore of backup SVHMJ9 completed.

c) 重启 openGauss 数据库后, 在 db1 数据库中执行 \d 命令, 发现数据库中有表 test1, 但不存在表 test2, 这时候数据库恢复到了创建 test1 之后进行增量备份的那一刻:



```
db1=# \d
```

```
                List of relations
 Schema | Name  | Type  | Owner  | Storage
-----+-----+-----+-----+-----
 public | person | table | opengauss | {orientation=row,compression=no}
 public | test1  | table | opengauss | {orientation=row,compression=no}
(2 rows)
```

d) 执行第二次增量恢复:
先删除整个数据库目录:

```
sudo rm -rf /var/lib/opengauss/data
```

```
gs_probackup restore -B /home/opengauss/physical/gs_bak --instance=gs_bak_inst -i SVH0AO
```

“第二次增量备份的文件 ID”

```
INFO: Syncing restored files to disk
INFO: Restored backup files are synced, time elapsed: 0
INFO: Restore of backup SVH0AO completed.
```

e) 重启 openGauss 数据库后, 在 db1 数据库中执行 \d 命令, 发现数据库中存在表 test1 和表 test2, 但不存在表 test3, 说明此时数据库恢复到了创建表 test2 之后进行增量备份的那一时刻:

```
db1=# \d
```

```
                List of relations
 Schema | Name  | Type  | Owner  | Storage
-----+-----+-----+-----+-----
 public | person | table | opengauss | {orientation=row,compression=no}
 public | test1  | table | opengauss | {orientation=row,compression=no}
 public | test2  | table | opengauss | {orientation=row,compression=no}
(3 rows)
```

2.3 数据库备份与恢复的练习

在 openGauss 中, 使用物理备份工具和逻辑备份工具:

- 1) 逻辑备份 employ 数据库, 以目录形式输出到 /home/opengauss/logic/team。
- 2) 逻辑备份 employees 员工表, 以自定义形式输出到 /home/opengauss/logic/employees.dump。
- 3) 对 openGauss 的所有数据库数据进行一次性逻辑备份, 并且以文本形式输出到 /home/opengauss/logic/database.bak。
- 4) 将 1) 导出的 employ 数据库目录恢复成原本的数据库。【先将数据库删除】
- 5) 将 1) 导出的 employees 员工表导入 employ 数据库中。【先将表删除】
- 6) 将 employ 数据库进行全量物理备份, 输出到目录 /home/opengauss/physic/gs_bak。
【注意先初始化备份目录、添加备份实例】
- 7) 将 employ 数据库进行增量物理备份, 输出到目录 /home/opengauss/physic/gs_bak。
- 8) 对 employ 数据库进行增量恢复。【先将整个数据库目录删除】